

TP2 : les data.frame et le chargement de fichiers

Introduction sur les `data.frame`

Les `data.frame` sont utilisés pour combiner plusieurs vecteurs qui concernent le même sujet pour faciliter leur traitement.

La manière la plus simple de générer un `data.frame` est de créer les vecteurs, puis de les combiner avec la fonction `data.frame`:

```
x <- 1:5
y <- c(4,5,8,2,4)
label <- c("a", "b", "c", "d", "e")
df <- data.frame(position = x, valeur = y, label = label)
df
#>   position valeur label
#> 1         1      4     a
#> 2         2      5     b
#> 3         3      8     c
#> 4         4      2     d
#> 5         5      4     e
```

On peut accéder aux noms de colonnes et modifier leurs valeurs avec la fonction `colnames()` :

```
colnames(df)
#> [1] "position" "valeur"  "label"
colnames(df) <- c("x", "y", "nom")
df
#>   x y nom
#> 1 1 4  a
#> 2 2 5  b
#> 3 3 8  c
#> 4 4 2  d
#> 5 5 4  e
```

On peut accéder aux noms de rangées et modifier leurs valeurs avec la fonction `rownames()` :

```
rownames(df)
#> [1] "1" "2" "3" "4" "5"
```

Charger les données d'un fichier

Les données qu'on souhaite traiter avec R vont régulièrement se retrouver sous le format de fichier texte. Pour le chargement de ces données, il est recommandé d'utiliser la fonction `read.table`. Les paramètres les plus utiles dans cette fonction:

- `header`: `TRUE` si le fichier contient un en-tête, sinon `FALSE`
- `sep`: Déterminer la valeur du séparateur (ex.: "`\t`" (tabulation), "`,`", "`;`", etc...)
- `stringsAsFactors`: depuis R 4.0, la valeur par défaut est `FALSE` (les textes restent des chaînes de caractères); inutile de la préciser

Par exemple, pour charger le fichier `df.tsv`, qui contient un en-tête et dont les valeurs sont séparées par des tabulations, on utilisera la commande suivante:

```
df_fichier <- read.table("df.tsv", header = TRUE, sep = "\t")
df_fichier
#>   x y nom
#> 1 1 4  a
#> 2 2 5  b
#> 3 3 8  c
#> 4 4 2  d
#> 5 5 4  e
```

À votre tour !

Ouvrez un script qui contiendra les étapes de votre travail. Notez les infos pertinentes en début de script en commentaire (`#`).

Chargez le fichier `/home/public/R/TP2.csv` avec la fonction `read.csv` et assignez-le à une variable:

```
objet1 <- read.csv('/home/public/R/TP2.csv')
```

Vous avez une description de l'objet dans le panneau *Environment*. Ceci correspond à un extrait de fichier VCF (*Variant Call Format*), un fichier qui indique des divergences entre des données de séquençage et un génome de référence.

0. Extrayez un *rectangle* de données (par exemple, 2 rangées sur trois colonnes) de l'objet1.
1. Extrayez les noms de colonnes (fonction `colnames`) et assignez à une variable.
2. Extrayez la colonne des identifiants de loci (`ID`) et assignez à une variable.
3. Trouvez le score qualité ('QUAL') pour le snp `rs189107123` (Astuce : utilisez le résultat précédent pour faire un vecteur logique où seul le snp visé est vrai en utilisant `==`).
4. Trouvez la position ('POS') du snp `rs4030331`.
5. Quel est le score qualité ('QUAL') moyen (la moyenne de la colonne 'QUAL') ?
6. Assignez la liste des snps dont la qualité est > 99 à la variable `good.snps`.

Et si... ?

Pour chaque question, **prédisez d'abord la réponse sans exécuter**, puis vérifiez dans R. Si le résultat vous surprend, cherchez pourquoi.

1. Et si on chargeait `df.tsv` sans `header = TRUE` : combien de lignes aurait le `data.frame`, et de quel type seraient les colonnes ?
2. Et si on cherchait le score qualité d'un SNP qui n'existe pas (`rs000`) avec la méthode de la question 3 : obtient-on une erreur ?
3. Et si on oubliait la virgule : `objet1[objet1$QUAL > 99]` . Que se passe-t-il, et pourquoi ?
4. Et si la colonne `QUAL` contenait une valeur manquante (`NA`) : qu'est-ce que cela changerait aux réponses des questions 5 et 6 ?

